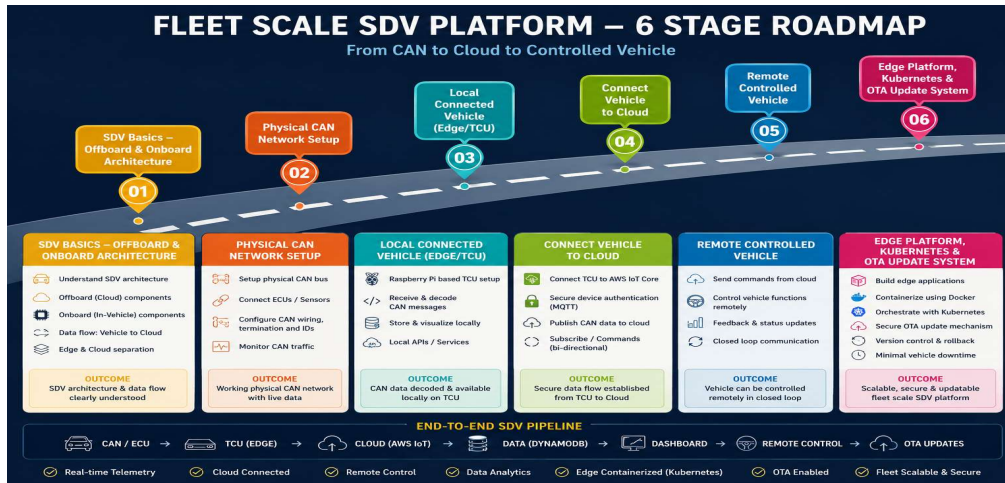


Roadmap - Learn SDV architecture from CAN to cloud — built at home



SDV - Architecture

The automotive industry is moving from a **distributed ECU architecture** and **domain architecture** toward a **zone-based architecture** with **centralized computing**, where software becomes the primary differentiator of vehicle functionality.

The key enablers are

- ✧ Centralized computing
- ✧ Zone controllers
- ✧ Remote-controlled edge nodes
- ✧ High-bandwidth vehicle networking (Ethernet)
- ✧ Intelligent power distribution
- ✧ OTA software updates
- ✧ Cybersecurity and functional safety

What is meaning of Software-Defined Vehicle

Traditionally we have Vehicle which have so many ECUs.

ECU 1 -> Door
ECU 2 -> Window
ECU 3 -> Seat
ECU 4 -> Lighting
ECU 5 -> HVAC

...

100+ ECUs

Each ECU owns its software.

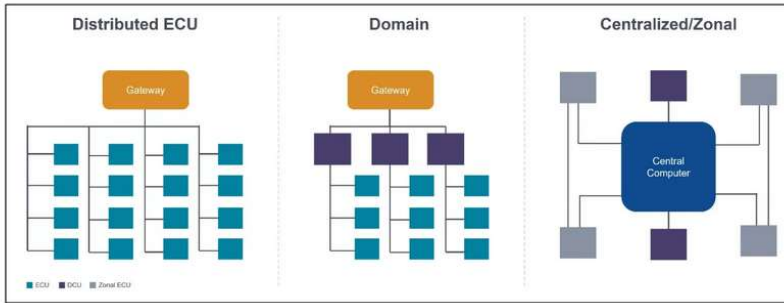
How these ECUs would be placed inside the vehicle?

Mainly 3 architecture are followed

Distributed ECUs

Domain Controllers unit

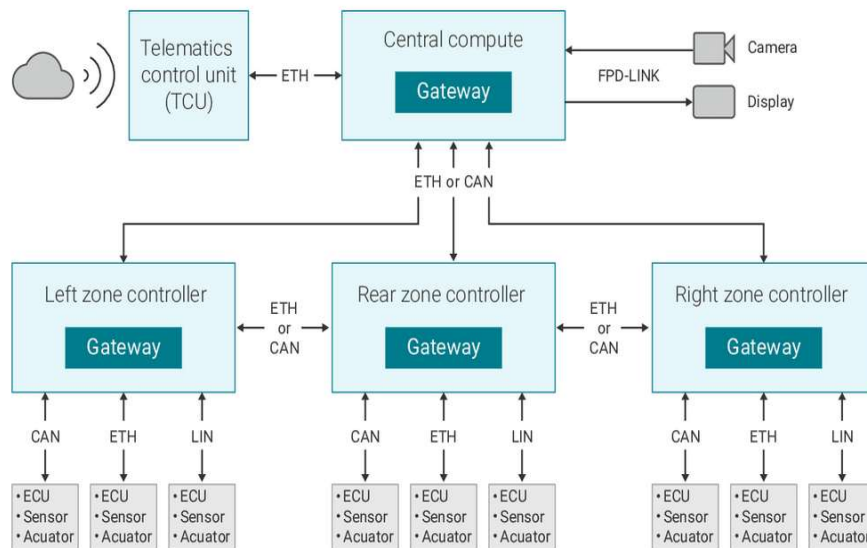
Centralized and zonal Architecture



[source](#)

The future Vehicles are moving towards Centralized/zonal Architecture

zone-architecture-in-vehicle-network



[source](#)

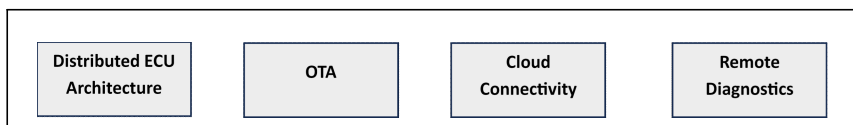
Software is centralized and hardware becomes more generic and reusable. This enables OTA updates, feature upgrades, and better scalability across vehicle platforms.

Now the Questions is?

SDV is about centralization of ECUs, portability of software over different hardware via hypervisors, service oriented communication, etc. and most important OTA. Seems like you assumed that SDV is about remotely controlling the vehicle, which is totally untrue.

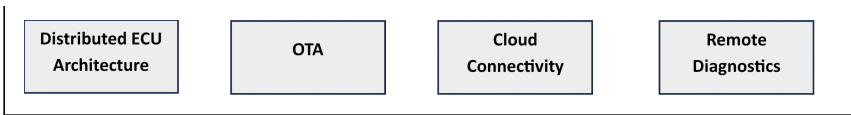
From <https://www.linkedin.com/feed/update/urn:li:activity:7466419974744576001/>

Why it cannot be a SDV architecture?



or



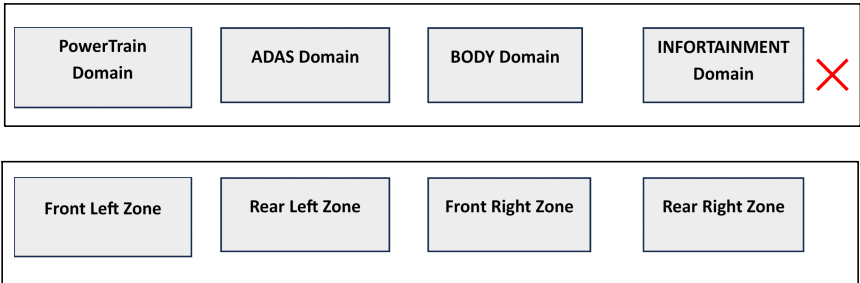


My takeaway is **SDV != Zonal Architecture**

But but all the OEMs are following Zonal architecture for SDV, why?

The Most Important Concept is Zonal Architecture (As rightly highlighted)

repeatedly emphasizes that the path to SDV is through **Zone Architecture**.
Instead of grouping Domain wise



Benefits:

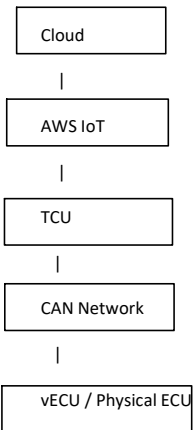
- ✧ Reduced wiring harness
- ✧ Reduced vehicle weight
- ✧ Lower system cost
- ✧ Easier scalability
- ✧ Centralized software deployment

Remote-Controlled Edge Nodes

This was the section I found most interesting for my SDV-at-home project.
introduces the idea of:

```

Central Computer
|
Ethernet
|
Zone Controller
|
Remote Edge Nodes
The edge nodes:
• Control actuators
• Read sensors
• Execute limited local logic
• Depend on centralized software control
This aligns closely with what you're building:
  
```



MY Raspberry Pi TCU effectively behaves like a simplified zone controller.

Ethernet Is the Backbone

clearly positions Ethernet as the future in-vehicle backbone.

Why?

Because modern SDVs need:

- ✧ OTA updates
- ✧ Camera streams
- ✧ Radar data
- ✧ LiDAR data
- ✧ Vehicle telemetry
- ✧ AI workloads

CAN alone cannot handle these bandwidth requirements (Because CAN speed is 500kbs if our binaries are in bigger size it will flash the ECUs but it will take more time)

IS CAN dead in SDV architecture ?

CAN/LIN ==>> Zone Controller ==> Automotive Ethernet ==>> Central Computer

Centralized Compute

divides computing into three layers:

1. Central Compute

High-performance processors.

Responsibilities:

- ✧ AI
- ✧ ADAS
- ✧ Vehicle software services
- ✧ OTA management
- ✧ Fleet services

2. Zone Controllers

Responsibilities:

- ✧ Aggregation
- ✧ Communication
- ✧ Local control

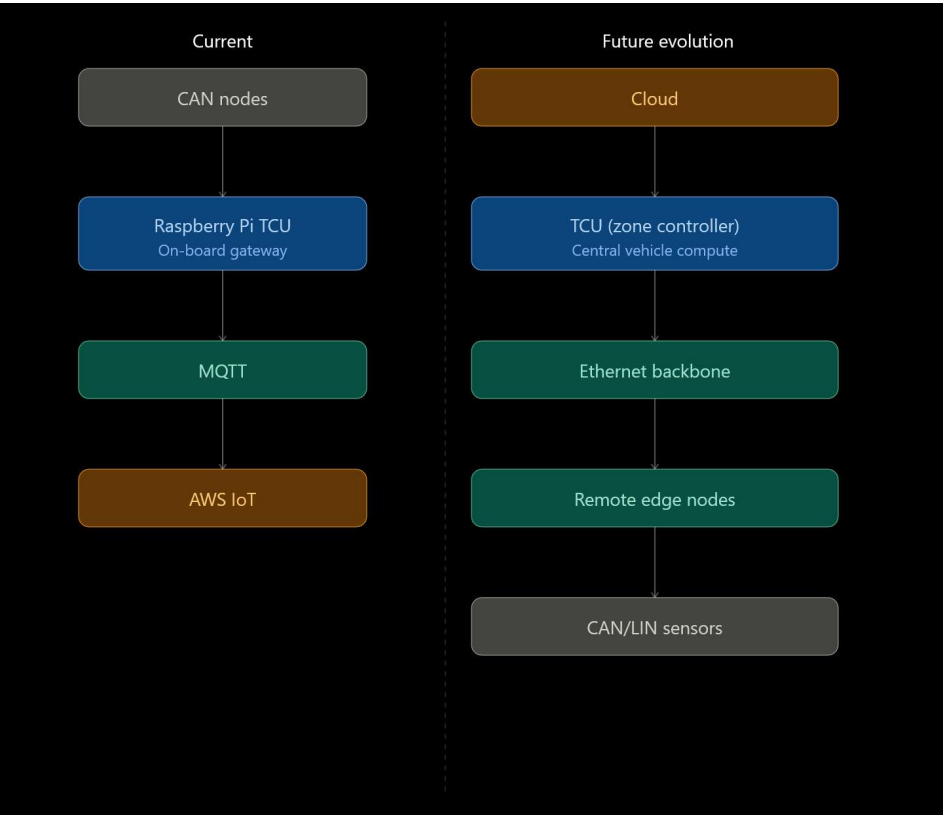
3. Edge ECUs

Responsibilities:

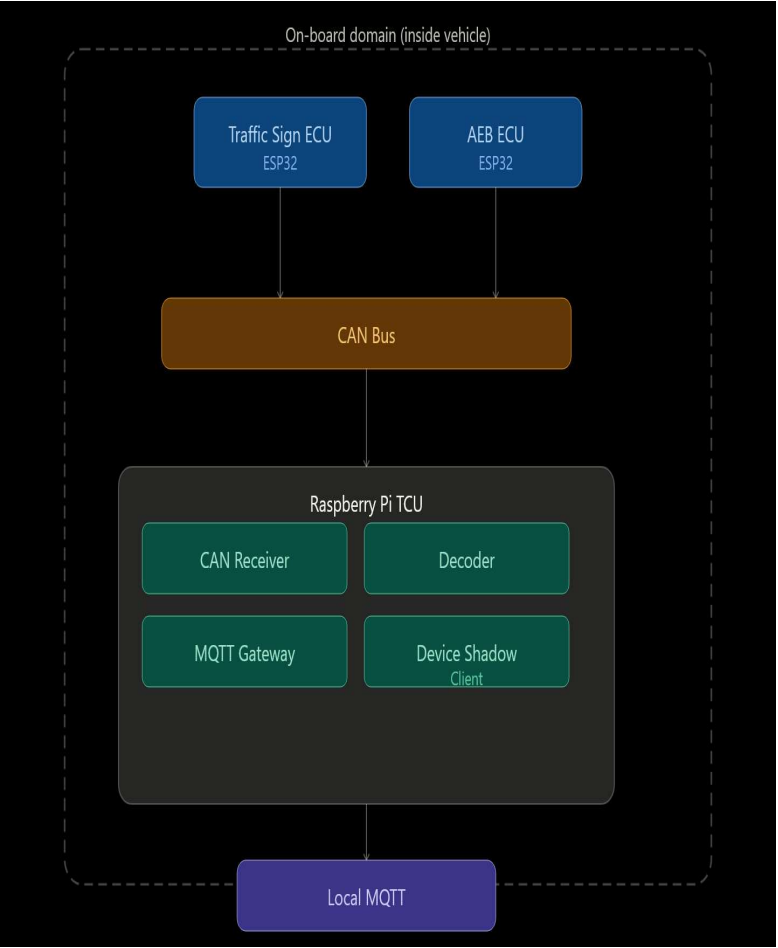
- ✧ Sensors
- ✧ Actuators
- ✧ Low-level control

What This Means for MY Home SDV Project

My roadmap is actually very close to the concepts what I described above.



which aligns strongly with SDV vision.



OTA
Observability
Digital Twin
Remote Diagnostics



My takeaway

The single most important message from SDV is

SDV is not primarily about cloud connectivity or OTA updates. It is about moving from many function-specific ECUs to a centralized compute + zonal architecture where software can be reused, updated, and scaled across vehicle platforms.

For building an SDV demonstrator at home, the concepts to focus on next are:

1. Zone Controller
2. Automotive Ethernet
3. Service-Oriented Communication
4. OTA Update Pipeline
5. Software Observability
6. Centralized Vehicle Services
7. Remote-Controlled Edge Nodes